MACHINE LEARNING PROJECT

Group Name :- LOSSLESS
Group No :- 11

Improving Road Damage Detection On Indian roads Using Synthetic Data Generation

---

Does adding synthetic road-damage images improve YOLO11n detection on real road images?

Name	Roll No.
Khushiba	25BDA054
Anubhav	25BDA013
Maneesh	25BDA063
Pranjal	25BDA085
Abhiram	25BDA121
Pavan	25BDA081
Vikas	25BDA126
Dhanashri	25BDA036
Amrita	25BDA012
Dhanush	25BDA037
Siddanna	25BDA115

Submitted to - Dr. Utkarsh Khaire

GITHUB_LINK :- LINK

Why automated road damage detection matters ?

Road damage causes accidents, vehicle damage, congestion and rising maintenance costs.

1

Manual inspection is slow

Inspection needs time, labour and regular monitoring across the road network.

2

Models lose accuracy

Detectors can weaken on unfamiliar roads, poor lighting, rainfall and different road surfaces.

3

Synthetic data as a remedy

Generated road-damage images may widen the range of damage appearances the model learns from.

OUR RESEARCH QUESTION -

Can synthetic road-damage images, combined with real road images, improve the accuracy and generalization of detection models for road damage in Indian conditions?

Objectives and hypotheses

Wetestedtheeffectofsyntheticdatabycomparingtwomodelson the same real test set.

Objectives

1. Review literature on road-damage detection and synthetic data
2. Collect and analyse Indian images from RDD2022
3. Build a real-only YOLO baseline with transfer learning
4. Generate synthetic images with image processing and LLM-assisted prompts
5. Train and compare real-only and real + synthetic models
6. Evaluate on unseen real road images, including challenging conditions

Null hypothesis (H0)

Adding synthetic road-damage images to the real training data does not improve detection performance on unseen real images.

Alternative hypothesis (H1)

Adding synthetic road-damage images to the real training data improves detection performance on unseen real images.

Dataset: Indian images from RDD2022

Road Damage Dataset 2022 covers six countries but this project uses the Indian subset.



Sample Indian road images - Red boxes are the original annotations (D20 alligator crack and D44 is an excluded class)

7,706

labelledimages used for training, validation and testing

5,779 / 770 / 1,157

train/validation/fixedtest(75%/10%/ 15%)

4

damage classes: longitudinal, transverse, alligator, pothole

~2,000

unlabelledtestimages kept separate and not used

Technologies and tools Used

Open-source Python stack, with training on Apple Silicon GPU acceleration.

Programming and environment

- Python 3.13.7
- VS Code and Jupyter Notebook
- Apple M4 laptop with MPS GPU acceleration

Deep learning and detection

- PyTorch 2.14.0
- Ultralytics 8.4.184 with YOLO11n
- Pretrained yolo11n.pt weights

Data processing

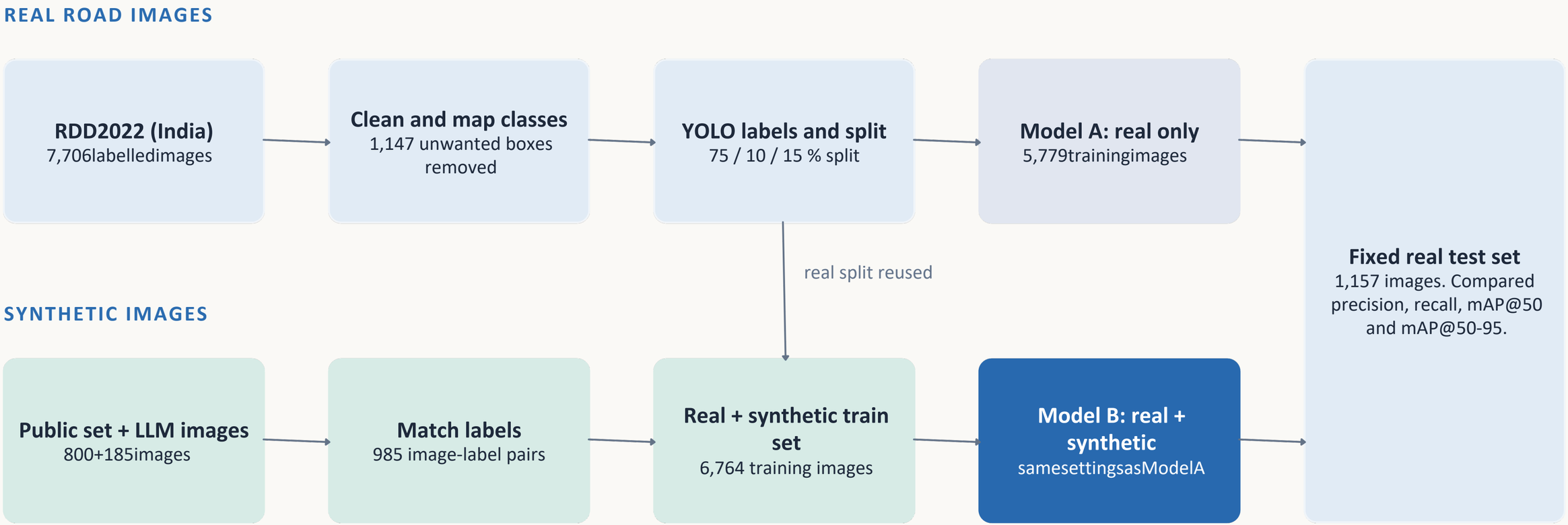
- pandas for tables and metrics
- xml.etree for Pascal VOC annotations
- pathlib, csv and shutil for file handling

Image handling and visualisation

- OpenCV (cv2) for reading images
- Matplotlib for bounding-box plots
- PyYAML for dataset configuration

System architecture: two training pipelines, one test set

Both models use the same validation and fixed real test split.



Model A and Model B use identical hyperparameters, so only the training images differ.

Data preprocessing: standardising four target classes

Fine-grained and non-structural labels were mapped or removed. No images were deleted.

Original label	Action	Objects
D01	Mapped to D00 (longitudinal)	179
D0w0	Corrected to D00 after visual check	1
D11	Mapped to D10 (transverse)	45
D43	Box removed (non-structural)	57
D44	Box removed (non-structural)	1,062
D50	Box removed (non-structural)	28
Total removed	Only the excluded <object> entries	**1,147**

Objects
8,203 to 7,056

Empty annotation files
3,921 to 4,387

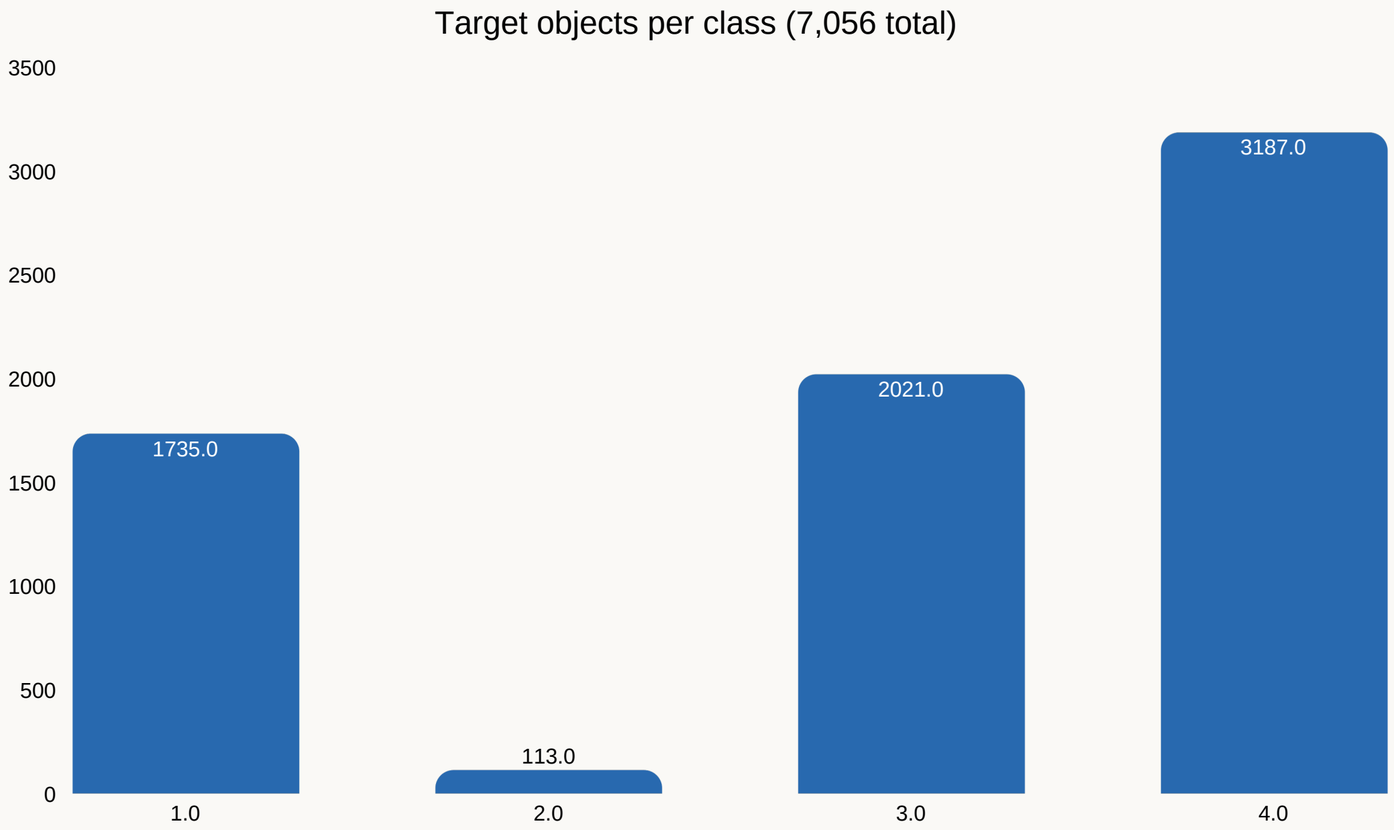
Images retained
7,706 of 7,706

Invalid XML files
0

Original XML files were backed up before cleaning. Images with only excluded boxes stay in the data as negative samples.

Exploratory analysis: class imbalance and empty images

The four classes are unevenly represented and more than half the images have no target damage.



74% of all boxes are potholes or alligator cracks (5,208 of 7,056).

1.6% of boxes are transverse cracks (113), the rarest class.

57% of images (4,387 of 7,706) have no target annotation and act as negative samples.

Methodology: a controlled comparison

Only the training images differ between Model A and Model B.

Setting	Model A: real only	Model B: real + synthetic
Training images	5,779 real	6,764 (5,779 real + 985 synthetic)
Validation images	770 real	770 real (identical)
Fixed test images	1,157 real	1,157 real (identical)
Starting weights	yolo11n.pt (pretrained)	yolo11n.pt (pretrained)
Epochs / patience	30 / 7	30 / 7
Image size / batch size	512 px / 16	512 px / 16
Random seed	42	42
Hardware	Apple M4, MPS	Apple M4, MPS
Observed training time	2.17 hours	3.55 hours

Model: YOLO11n with transfer learning

A lightweight single-stage detector, fine-tuned from pretrained weights.

- **Object detection:** predicts the damage class and a bounding box in one pass.
- **Transfer learning:** starts from pretrained yolo11n.pt; 493 of 499 weight items transferred.
- Fine-tuned on four road-damage classes, so the output head predicts four classes.
- Chosen as the proposed model; its small size makes repeated training on a laptop practical.

2.6 M

parameters in the trained model

6.4

GFLOPsperforward pass

4

output classes

512 px

square input image size

Synthetic data: two sources, 985 images

A public computer-vision set and LLM-assisted images, each matched to YOLO labels.

1 Public computer-vision dataset

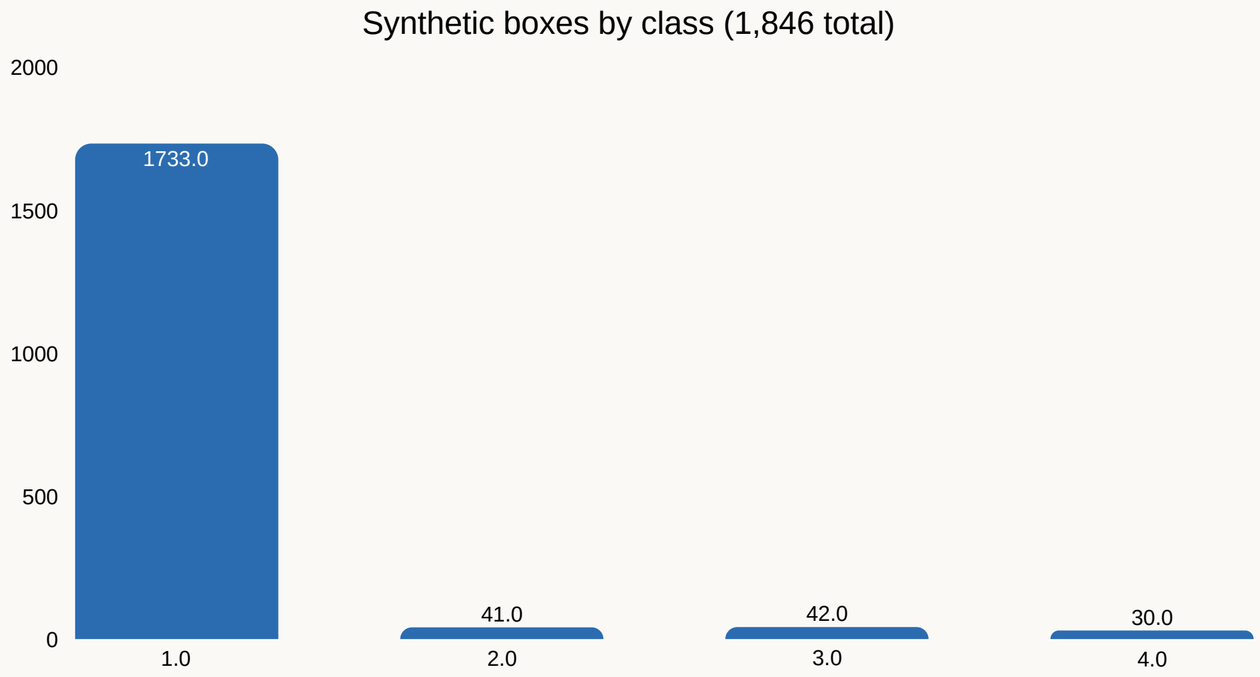
800 images. All 800 label files matched by numeric ID.

Source: Figshare, potholes dataset with YOLO annotations.

2 LLM-assisted generated images

185 images. All 185 label files matched by numeric ID.

Prompts described damage type, road surface, lighting, camera angle and environment.



94% of synthetic boxes are longitudinal cracks, so this data mainly adds one class.

Implementation: key pipeline steps

Simplified excerpts from the notebook; full code is in Project.ipynb.

1. Clean XML labels: map classes and drop excluded boxes
2. Convert to YOLO format with normalised box coordinates
3. Create a seeded train / validation / test split
4. Match synthetic images to their label files
5. Train Model A and Model B with identical settings
6. Evaluate both on the same fixed real test set

Label mapping

```
CLASS_MAP = {"D01": "D00", "D0w0": "D00",
             "D11": "D10", "D20": "D20", "D40": "D40"}
DROP = {"D43", "D44", "D50"}
```

Training (Model B)

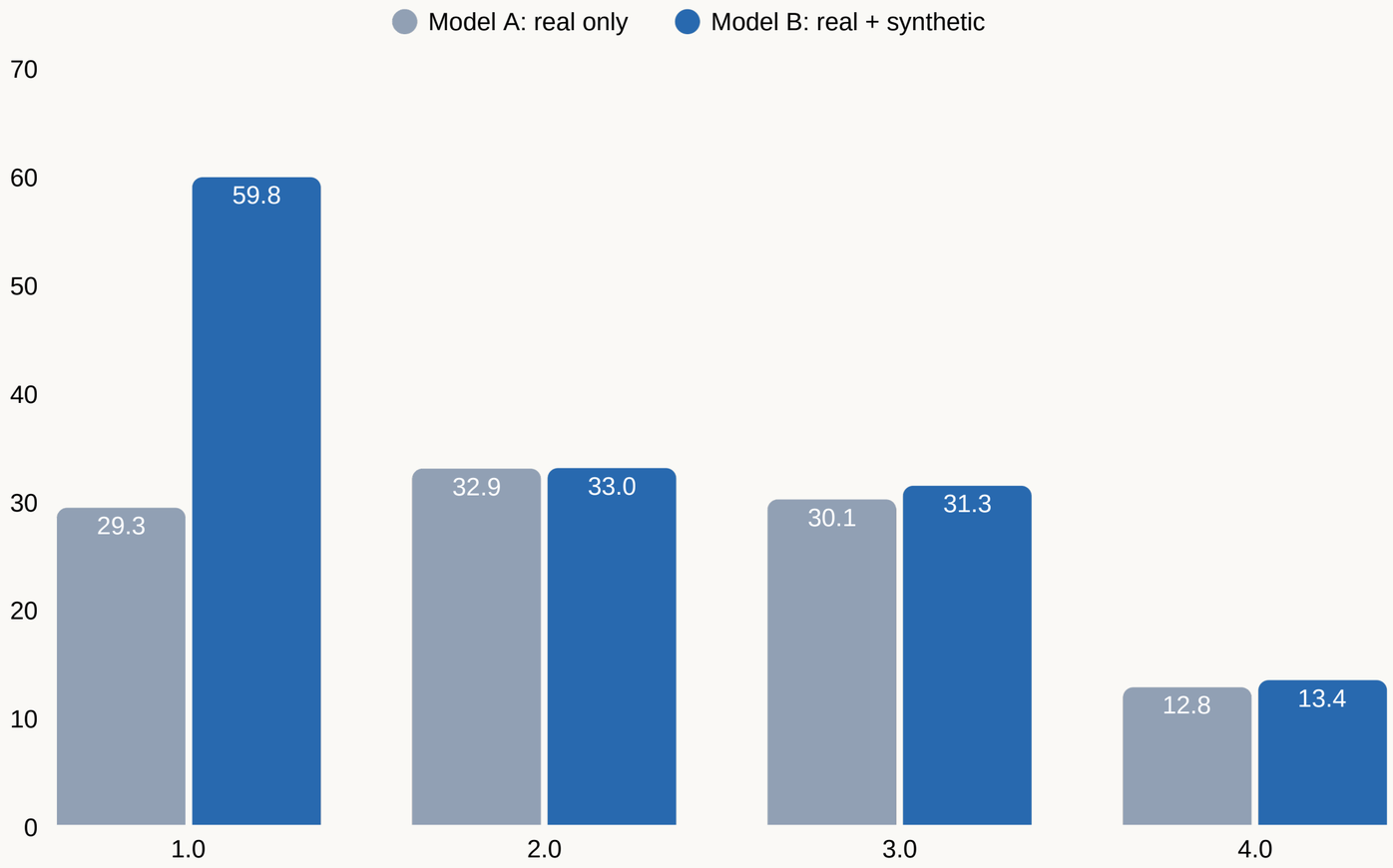
```
model = YOLO("yolo11n.pt")
model.train(data="data_real_synthetic.yaml", epochs=30,
            patience=7, imgsz=512, batch=16,
            device="mps", seed=42)
```

Evaluation on fixed test set

```
metrics = model.val(data=test_yaml, split="test",
                    imgsz=512, batch=16, device="mps")
metrics.box.mp, metrics.box.mr, metrics.box.map50
```

Experimental results: precision rose, recall held steady

Test-set comparison on the same 1,157 real images (1,108 damage objects).



Metric	Model A	Model B	Change
Precision	29.31%	59.80%	**+30.50 pp**
Recall	32.92%	32.97%	**+0.06 pp**
mAP@50	30.08%	31.34%	**+1.26 pp**
mAP@50-95	12.75%	13.41%	**+0.66 pp**

pp = percentage points. Each model was trained once (seed 42).

Evaluation metrics: what each number tells us

Detection is judged on both whether damage is found and where the box is placed.

Precision

Of the damage boxes the model predicts, the share that are correct. Higher precision means fewer false alarms.

Recall

Of all real damage in the test images, the share the model finds. Higher recall means fewer missed defects.

mAP@50

Average precision across classes when a predicted box counts as correct at IoU of 0.50 or more.

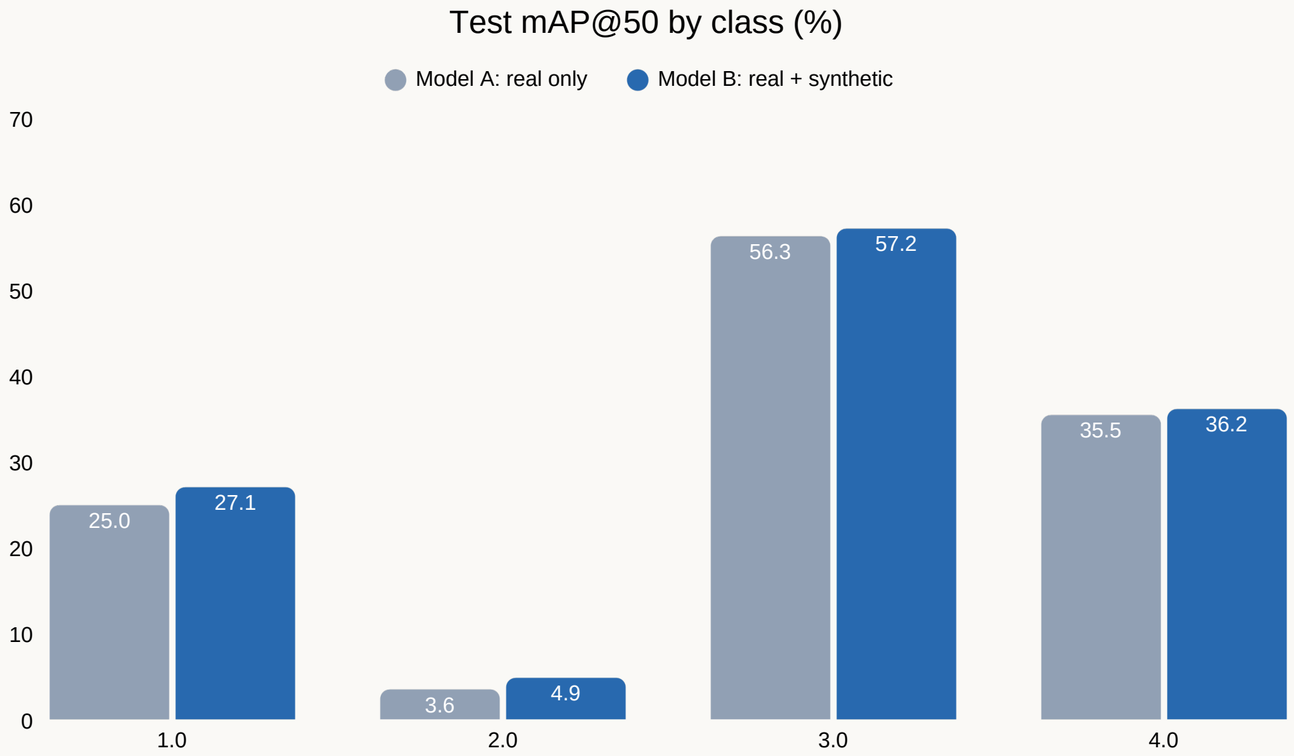
mAP@50-95

The same average, taken over IoU thresholds from 0.50 to 0.95. It is stricter and rewards precise box placement.

IoU (intersection over union) measures how much a predicted box overlaps the true box: 1.0 is a perfect match.

Analysis: where the gains came from

Precision drove most of the change; recall and per-class gains were small.



- Precision nearly doubled, from 29.31% to 59.80%.
- Recall did not move (32.92% vs 32.97%), so the share of damage found is unchanged.
- mAP gains are small: +1.26 pp at mAP@50 and +0.66 pp at mAP@50-95.
- Every class improved slightly, but transverse cracks have only 12 test objects.
- On validation, mAP@50 was equal (31.9%) and mAP@50-95 rose from 13.6% to 14.0%.

Preliminary evidence: directionally positive, but no significance test or repeated runs yet.

Challenges and limitations

Several factors limit how strongly the results can be interpreted.

1

Class imbalance

Transverse cracks have 113 objects in total and only 12 in the test set.

2

Skewed synthetic data

94% of synthetic boxes are longitudinal cracks, so the added data mostly reinforces one class.

3

Limited statistical evidence

One training run per model (seed 42). No confidence intervals or significance test yet.

4

Many empty images

57% of images have no target labels. Some may still show unlabelled damage.

5

Manual label correction

One D0w0 label was corrected after visual inspection, not by an automated rule.

6

Condition-specific results

The notebook does not yet report results for rain, low light or different road surfaces.

Conclusion

Preliminaryevidencethatsynthetic data can help precision on real Indian road images.

What we achieved

- Built a reproducible pipeline from raw RDD2022 annotations to YOLO training data.
- Trained a real-only baseline and a real + synthetic model under identical settings.
- Raised test precision from 29.31% to 59.80% on the same fixed real test set.
- Verified matching for all 985 synthetic image-label pairs.

References and acknowledgements

A Multi-national Image Dataset for Automatic Road Damage Detection (RDD2022). doi:10.1002/gdj3.260

Toward Domain Adaptive Road Damage Detection Across Different Countries. doi:10.1109/TITS.2022.3221067

Road Damage Detection with Bounding Box - doi:10.1049/ipr2.12940

RDD2022 dataset on Figshare - doi:10.6084/m9.figshare.21431547.v1

Potholes dataset with YOLO annotations (Figshare) figshare.com/articles/figure/Potholes_dataset_with_YOLO_annotations/21214400

Official RoadDamageDetector repository. github.com/sekilab/RoadDamageDetector
Ultralytics YOLO11 (Ultralytics 8.4.184), used for training and evaluation